

Runtime Polymorphism in C++ — Complete Revision Notes

1. What is Polymorphism?

The word **Polymorphism** means "**many forms.**"

In C++, polymorphism allows the **same function call** to behave differently depending on the object involved.

There are two major types:

1. Compile-Time Polymorphism (Static Polymorphism)

The compiler decides which function to call **while compiling the program.**

Examples:

- Function Overloading
- Operator Overloading

2. Runtime Polymorphism (Dynamic Polymorphism)

The compiler postpones the final decision until **the program is actually running.**

This is achieved using:

- Inheritance
- Virtual Functions
- Base Class Pointer or Base Class Reference

2. Why My First Understanding Was Confusing

Initially, I thought:

```
Parent p;  
Child c;  
  
p.getInfo();  
c.getInfo();
```

Since the compiler already knows:

- `p` is a Parent

- `c` is a Child

it immediately decides which function to call.

So I wondered:

"If the compiler already decides everything during compilation, then why is this called Runtime Polymorphism?"

The answer is:

This example is NOT runtime polymorphism.

Even if `getInfo()` is declared `virtual`, calling functions directly on objects does not require runtime decision-making because the compiler already knows the object's exact type.

3. The Real Runtime Polymorphism

The important code is:

```
Parent* ptr = &c;  
  
ptr->getInfo();
```

This is where runtime polymorphism actually happens.

4. Step-by-Step Process

Step 1

Create the classes.

```
class Parent  
{  
public:  
    virtual void getInfo()  
    {  
        cout << "Parent";  
    }  
};  
  
class Child : public Parent  
{  
public:
```

```
void getInfo() override
{
    cout << "Child";
}
};
```

The keyword `virtual` tells the compiler:

"Do not always decide this function call during compilation. The actual object should decide."

Step 2

Create objects.

```
Parent p;
Child c;
```

Now two different objects exist.

One Parent object.

One Child object.

Step 3

Create a base-class pointer.

```
Parent* ptr;
```

At this point, the pointer points nowhere.

It is only capable of pointing to:

- Parent objects
- Any class derived from Parent

It cannot point to unrelated classes.

This is because of inheritance.

Step 4

Assign it.

```
ptr = &c;
```

Now the pointer stores the address of the Child object.

Important:

The pointer type is still

```
Parent*
```

But the actual object is

```
Child
```

These are different concepts.

This difference is the heart of runtime polymorphism.

5. Static Type vs Dynamic Type

This is the most important concept.

Static Type

Static type is the type known from the variable declaration.

Example:

```
Parent* ptr;
```

The static type is:

```
Parent*
```

The compiler always knows this.

Dynamic Type

Dynamic type means the actual object currently stored in memory.

If

```
ptr = &p;
```

Dynamic type:

Parent

If

```
ptr = &c;
```

Dynamic type:

Child

The compiler cannot always rely only on the static type because the dynamic type may vary while the program runs.

6. What Does the Compiler Know?

The compiler definitely knows:

- `ptr` is a Parent pointer.
- `getInfo()` is virtual.

However, the compiler does not permanently bind the function call to `Parent::getInfo()`.

Instead, it generates code that says:

"When this line executes, inspect the actual object and then call the correct version."

This is called **dynamic dispatch**.

The compiler is not "confused." It is intentionally following the rules of virtual functions.

7. Why Not Decide During Compilation?

Imagine this:

```
Parent* ptr;  
  
if(userChoice == 1)  
    ptr = new Parent();  
else  
    ptr = new Child();  
  
ptr->getInfo();
```

The compiler cannot predict:

- What the user will choose.
- Which object `ptr` will point to.

Only when the program is actually running will that information exist.

Therefore, the decision is delayed until runtime.

8. What Happens During Runtime?

When execution reaches

```
ptr->getInfo();
```

the runtime mechanism checks:

"What object is this pointer actually pointing to?"

If it points to a Parent object:

```
Parent::getInfo()
```

is executed.

If it points to a Child object:

```
Child::getInfo()
```

is executed.

This is runtime polymorphism.

9. My Mathematical Analogy

This helped me understand the concept.

Suppose I have:

$$x + 5$$

Without knowing `x`, I cannot produce the final answer.

If

$$x = 2$$

Answer:

7

If

$$x = 100$$

Answer:

105

The formula is fixed.

The value changes.

Similarly,

```
ptr->getInfo();
```

is like the formula.

The object currently referenced by `ptr` is like the value of `x`.

Only after the actual object is known can the correct function be chosen.

10. Does the Compiler Know the Address?

Sometimes yes.

For simple code like

```
Parent* ptr = &c;
```

the compiler can see that `ptr` is assigned the address of `c`.

However, runtime polymorphism is **not about the address itself**.

It is about **which object's virtual function should be executed**.

In larger programs, the object may come from:

- Another function
- User input
- File reading
- Network request
- Factory functions
- Dynamic allocation

Therefore, C++ uses the same runtime mechanism consistently.

11. Why Use Parent *Instead of* Child?

If we write

```
Child* ptr = &c;
```

the compiler already knows:

- pointer type = `Child*`
- object type = `Child`

There is no need for runtime dispatch.

But with

```
Parent* ptr = &c;
```

the same pointer can refer to many different derived objects, making polymorphism possible.

12. How Does C++ Do This Internally?

When a class has virtual functions, the compiler secretly creates:

- A virtual table (vtable)
- A hidden pointer inside each object (vptr)

When

```
ptr->getInfo();
```

is executed, C++ roughly performs:

1. Follow `ptr` to the object.
2. Read the object's hidden `vptr`.
3. Find the correct function in the `vtable`.
4. Call that function.

The programmer never writes this code. The compiler generates it automatically.

13. Key Takeaways

- Runtime polymorphism requires inheritance.
 - Runtime polymorphism requires a virtual function.
 - Runtime polymorphism works through a base-class pointer or reference.
 - Direct object calls are not examples of runtime polymorphism.
 - The pointer's static type and the object's dynamic type can be different.
 - Virtual functions tell the compiler to perform dynamic dispatch.
 - The final function is selected according to the actual object during program execution.
 - This behavior is implemented internally using `vptr` and `vtable`.
-

Final One-Line Definition

Runtime polymorphism is the C++ feature in which a virtual function call made through a base-class pointer or reference is resolved according to the actual (dynamic) object at runtime rather than solely by the pointer or reference's declared (static) type.